Aula 04 - Abstração e Relacionamentos entre Objetos

Associação, Agregação, Composição e Diagramas de Classes UML

Prof. Dr. Welington Galvão Rodrigues

Análise e Desenvolvimento de Sistemas - ADS
IFMA — Campus Grajaú

02 de setembro de 2026

INSTITUTO FEDERAL
Maranhão

INSTITUTO FEDERAL
Maranhão

Agenda

1. Introdução
2. Abstração
3. Relacionamentos: Associação
4. Relacionamentos: Agregação e Composição
5. Diagramas de Classes UML Básicos
6. Miniprojeto: Pedido e ItemPedido
7. Exercício em Duplas
8. Síntese

INSTITUTO FEDERAL
Maranhão



Introdução

Retomada da Aula 03

- O que é encapsulamento e por que atributos públicos são um problema?
- Qual a diferença entre um getter e um setter?
- Por que validar dados dentro de um setter?

Agenda de Hoje

Até agora, trabalhamos com classes **isoladas** — `Televisor`, `ContaBancaria`, `Pessoa`.

> **Mas sistemas reais...**
> ...são compostos por objetos que se **relacionam** entre si: um pedido tem itens, uma turma tem alunos, um professor leciona uma disciplina.

Hoje: como **modelar** (abstração) e como **representar e implementar** esses relacionamentos.



Abstração

Uma Pergunta para Começar

Pergunta disparadora

Se eu pedir para modelarem uma classe `Pessoa` para o sistema de matrícula da faculdade, e depois para uma clínica veterinária (donos de animais), o resultado seria o mesmo?

Os atributos relevantes mudam conforme o **problema**, mesmo que a entidade do mundo real seja "a mesma pessoa".

O que é Abstração?

Abstração é o processo de identificar as **características essenciais** de uma entidade **para um problema específico**, ignorando detalhes irrelevantes naquele contexto.

A classe, em Java, é o **resultado** desse processo — um molde que já vem com a escolha de "o que importa" feita pelo programador.

Abstração x Encapsulamento

Abstração

- Decide **o quê** modelar
- Nível de design/análise
- Acontece antes de escrever a classe

Encapsulamento

- Decide **como proteger** o que foi modelado
- Nível de implementação
- Acontece dentro da classe já definida

A Mesma Entidade, Duas Abstrações

Entidade do mundo real: **"Livro"**.

Sistema de biblioteca	**Sistema de uma gráfica**
(o que importa: emprestar)	(o que importa: imprimir)
`titulo`, `autor`, `isbn`	`numeroDePaginas`, `tipoDePapel`
`disponivelParaEmprestimo`	`custoDeImpressao`

Abstração 1: Sistema de Biblioteca

```java
public class Livro {
    private String titulo, autor, isbn;
    private boolean disponivelParaEmprestimo;
}
```

Só o que importa para **emprestar/devolver** um livro.

Abstração 2: Sistema de uma Gráfica

```java
public class ItemDeImpressao {
    private int numeroDePaginas;
    private String tipoDePapel;
    private double custoDeImpressao;
}
```

Discussão

Por que a classe da biblioteca não precisa saber o tipo de papel? E por que a da gráfica não precisa saber se o livro está emprestado?

Síntese: Abstração

Abstração é escolher o **nível certo de detalhe**: nem de menos (faltando algo essencial), nem de mais (sobrecarregando a classe com informação irrelevante para o problema).



Relacionamentos: Associação

Objetos Não Vivem Sozinhos

Pergunta disparadora
As classes que vimos até agora existem sozinhas no sistema, ou costumam se referir a outras classes?

Raramente um objeto existe totalmente isolado — um `Professor` está ligado a uma `Disciplina`, um `Pedido` está ligado a um `Cliente`.

Associação

Associação é o relacionamento mais simples: uma classe **conhece** ou **usa** outra, geralmente mantendo uma referência a ela como atributo.

```java
public class Disciplina {
    private String nome;
    private Professor professor;    // associacao
}
```

As duas classes podem existir de forma **totalmente independente** uma da outra.



Relacionamentos: Agregação e Composição

Dois Casos Especiais de Associação

Pergunta disparadora

Uma `Turma` tem `Alunos`. Se a turma for encerrada, os alunos deixam de existir? E um `Pedido` tem `ItensDoPedido` — se o pedido for cancelado, os itens ainda fazem sentido sozinhos?

As respostas são diferentes — é isso que diferencia **agregação** de **composição**.

Agregação — Vínculo Fraco

A **parte** pode existir **independentemente** do **todo**. Remover o todo não destrói as partes.

```java
public class Turma {
    private String codigo;
    private List<Aluno> alunos;    // agregacao
}
```

Se a `Turma` for removida, cada `Aluno` continua existindo no sistema.

INSTITUTO FEDERAL
Maranhão

AGREGAÇÃO EM POO

WHOLE

DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO

DEPARTAMENTO
(Todo)

Contém:
- Professores

DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO

Objeto Professor Independente

Ciclo de Vida Independente

Departamento ◇—— **Professor**

PROFESSORES (Partes)



- Objeto Professor (Parte)
- Ciclo de Vida Independente

Ana (Professora)



- Objeto Professor (Parte)
- Ciclo de Vida Independente

Lucas (Professor)



- Cafia Professora Trabalhar
- Ciclo de Vida Independente

Sofia (Professora)

A relação 'Todo-Parte' fraca. As partes (Professores) continuam existindo mesmo se o todo (Departamento) for destruído.

Composição — Vínculo Forte

A **parte não existe** sem o **todo**. Se o todo é destruído, as partes também são.

```java
public class Pedido {
    private int numero;
    private List<ItemPedido> itens = new ArrayList<>();  // composicao
}
```

Um `ItemPedido` avulso, sem nenhum `Pedido`, não tem sentido no domínio do problema.

COMPOSIÇÃO EM POO

VEÍCULO (Todo)

[Veículo]
Tem:
- Motor
- Transmissão
- Rodas

Diamante Preenchido = Composição

Dramantido = Composição (Relação Forte)

MOTOR (Parte)



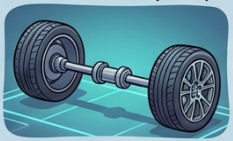
MOTOR (Parte)

- Objeto Parte Dependente. Não tem existência independente.
- Ciclo de Vida Amarrado ao Veículo.



TRANSMISSÃO (Parte)

- Transmissão (Parte)
- Ciclo de Vida Amarrado ao Veículo.



RODAS (Parte)

- RODAS (Parte)
- Ciclo de Vida Amarrado ao Veículo.

O "Todo" controla a criação e destruição das partes.
Se o Veículo for destruído, suas partes deixam de existir.

[Veículo] ◆→ [Parte]

A Composição é uma relação "Has-a" forte e exclusiva. As partes (e.g., o motor) são criadas e destruídas juntamente com o todo (o veículo) e não têm existência independente.

Associação x Agregação x Composição

Aspecto	Associação	Agregação	Composição
Força do vínculo	Fraca	Média ("tem um")	Forte
Parte existe sem o todo?	N/A	Sim	Não
Exemplo	Professor/Disciplina	Turma/Aluno	Pedido/ItemPedido

A pergunta-chave: **"a parte tem sentido sem o todo?"**

Reforçando com Código: Agregação

```java
// A lista de alunos pode ser passada de fora,
// alunos ja preexistem no sistema
public class Turma {
    private List<Aluno> alunos;

    public void matricular(Aluno aluno) {
        alunos.add(aluno);
    }
}
```

Reforçando com Código: Composição

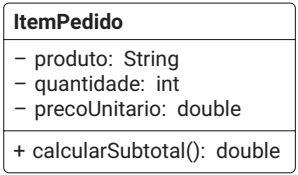
```java
// O Pedido cria seus proprios itens --
// eles nao existem antes dele
public class Pedido {
    private List<ItemPedido> itens = new ArrayList<>();

    public void adicionarItem(String produto, int qtd, double preco) {
        itens.add(new ItemPedido(produto, qtd, preco));
    }
}
```



Diagramas de Classes UML Básicos

A Caixa de Classe



Três compartimentos: **nome**, **atributos**, **métodos**.

O sinal "**–**" indica `private`; o sinal "**+**" indica `public`.

Notação de Relacionamento

Relacionamento	Notação
Associação	linha simples
Agregação	linha com losango **vazado** no lado do "todo"
Composição	linha com losango **preenchido** no lado do "todo"

Multiplicidade



- `1` — exatamente um
- `0..1` — zero ou um
- `*` — zero ou muitos
- `1..*` — um ou muitos

Prática no Quadro

Vamos desenhar juntos

`Pedido` e `ItemPedido` (composição) e `Turma` e `Aluno` (agregação).

Ferramentas

Papel/caneta, draw.io (biblioteca UML pronta) ou **Mermaid** (`classDiagram`, renderiza no Obsidian). Aprofundamento em `Materiais/Aula EXTRA - UML/`.



Miniprojeto: Pedido e ItemPedido

Live Coding: ItemPedido

```java
public class ItemPedido {
    private String produto;
    private int quantidade;
    private double precoUnitario;

    public ItemPedido(String produto, int quantidade, double
    precoUnitario) {
        this.produto = produto;
        this.quantidade = quantidade;
        this.precoUnitario = precoUnitario;
    }
}
```

INSTITUTO FEDERAL
Maranhão

Live Coding: `ItemPedido` — Subtotal

```java
public double calcularSubtotal() {
    return quantidade * precoUnitario;
}

public String getProduto() {
    return produto;
}
```

INSTITUTO FEDERAL
Maranhão

Live Coding: `Pedido` — Composição

```java
public class Pedido {
    private int numero;
    private List<ItemPedido> itens = new ArrayList<>();

    public Pedido(int numero) {
        this.numero = numero;
    }

    public void adicionarItem(String produto, int quantidade, double
     preco) {
        itens.add(new ItemPedido(produto, quantidade, preco));
    }
}
```

INSTITUTO FEDERAL
Maranhão

Pergunta para a Turma

INSTITUTO FEDERAL
Maranhão

Discussão
Por que `adicionarItem` recebe os dados soltos (produto, quantidade, preço) em vez de receber um `ItemPedido` já pronto?

Isso tem a ver com o que vimos sobre **composição** — o `ItemPedido` não existe fora do `Pedido`.

Live Coding: `calcularTotal()`

```java
public double calcularTotal() {
    double total = 0;
    for (ItemPedido item : itens) {
        total += item.calcularSubtotal();
    }
    return total;
}

public int getQuantidadeDeItens() {
    return itens.size();
}
```

INSTITUTO FEDERAL
Maranhão

Testando o Miniprojeto

```java
Pedido pedido = new Pedido(1001);

pedido.adicionarItem("Caderno", 3, 12.50);
pedido.adicionarItem("Caneta", 5, 2.00);

System.out.println("Itens: " + pedido.getQuantidadeDeItens());
System.out.println("Total: R$ " + pedido.calcularTotal());
```

Testando o Miniprojeto — Saída

Saída:

```
Itens:  2
Total:  R$ 47.5
```



Exercício em Duplas

Exercício em Duplas

Estendam o miniprojeto `Pedido`/`ItemPedido`

- `removerItem(String produto)`, removendo o primeiro item correspondente;
- `listarItens()`, imprimindo todos os itens com seus subtotais;
- Atributo `percentualDesconto` + `calcularTotalComDesconto()`.

Entrega da Lista de Exercícios 3

Entrega

Lista de exercícios 3 — abstração, associação, agregação, composição e diagramas de classes UML básicos. Compõe a nota de atividades da 1ª Etapa.



Síntese

Conceitos-Chave desta Aula

- Abstração: características essenciais para um problema
- Associação: conhece/usa, sem dependência
- Agregação: parte existe sem o todo
- Composição: parte não existe sem o todo
- Caixa de classe UML: nome, atributos, métodos
- Losango vazado/preenchido + multiplicidade

Próximas Aulas

Aula 05 — 02/09/2026 (quarta-feira)

Prática integradora: laboratório de resolução de problemas aplicando classes, objetos, encapsulamento e abstração — preparação para a avaliação da 1ª Etapa.

Encaminhamentos

Entregar a lista de exercícios 3 ao final da aula. Finalizar em casa a extensão do miniprojeto, se necessário. Revisar as Aulas 01 a 04 para a avaliação.

Obrigado!

Dúvidas ou sugestões?

welington.rodrigues@ifma.edu.br



INSTITUTO
FEDERAL
Maranhão

Aula 04 - Abstração e Relacionamentos entre Objetos

Associação, Agregação, Composição e Diagramas de Classes UML

Prof. Dr. Welington Galvão Rodrigues

Análise e Desenvolvimento de Sistemas - ADS
IFMA — Campus Grajaú

02 de setembro de 2026

INSTITUTO FEDERAL
Maranhão

INSTITUTO FEDERAL
Maranhão